

Experiment-6

Recurrent Neural Network

Aim

To study and implement a vanilla Recurrent Neural Network (RNN) for sequence modeling by training a simple character-level language model on a small excerpt of the Tiny Shakespeare dataset, and to understand sequence unrolling and hidden-state evolution through interactive visualization for a short sample sequence.

Theory

Recurrent Neural Networks (RNNs) are a class of deep learning models specifically designed to process and model sequential data. RNNs originated as models inspired by biological neural systems and were initially explored within cognitive science and neuroscience. Over time, they were adopted by the machine learning community as powerful tools for modeling sequential data.

Unlike traditional feedforward neural networks, which assume inputs to be independent of one another, RNNs incorporate recurrent connections that allow information to persist across time steps. These recurrent connections can be visualized as cycles in the network architecture, enabling the model to retain and utilize information from previous inputs while processing the current input.

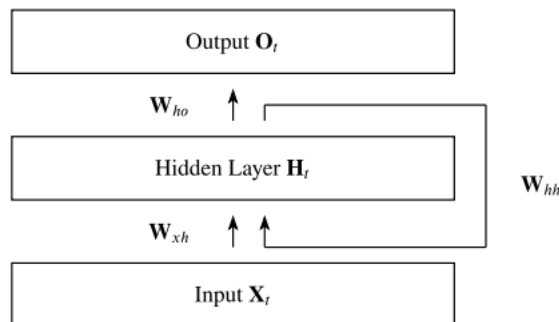


Fig. 1. Visualization of Recurrent Neural Network

(Source: Robin M. Schmidt, "Recurrent Neural Networks (RNNs): A Gentle Introduction and Overview," arXiv:1912.05911v1, 2019)

Mathematical Representation of Recurrent Neural Network

An RNN processes an input sequence one element at a time while maintaining a hidden state that captures information from previous time steps. The process begins with an initial hidden state H_0 , which is typically initialized to a vector of zeros or learned as a model parameter. At each subsequent time step $t \geq 1$, the hidden state H_t is updated using the current input X_t and the hidden state from the previous time step H_{t-1} .

The hidden state update is given by:

$$H_t = \varphi_h(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

where:

- X_t is the input at time step t
- H_t is the hidden state at time step t
- W_{xh} is the input-to-hidden weight matrix
- W_{hh} is the hidden-to-hidden (recurrent) weight matrix
- b_h is the bias vector
- $\varphi_h(\cdot)$ is the activation function (typically tanh or ReLU)

The output O_t at time step t is computed as:

$$O_t = \varphi_o(H_t W_{ho} + b_o)$$

where:

- W_{ho} is the hidden-to-output weight matrix
- b_o is the output bias
- $\varphi_o(\cdot)$ is the output activation function (e.g. softmax for classification)

This process is repeated across all time steps, with the same parameters $\{W_{xh}, W_{hh}, W_{ho}, b_h, b_o\}$ shared throughout the sequence, enabling the RNN to model temporal dependencies effectively.

Time Unrolling and Parameter Sharing

Upon initial observation, the presence of cycles may appear to contradict the feedforward nature of neural networks, where computation flows strictly in one direction. However, RNNs resolve this apparent ambiguity through a precise formulation: the network is unrolled across time steps. In the unrolled representation, the RNN is transformed into a sequence of identical feedforward networks, one for each time step, where the same set of parameters is shared across all steps. This parameter sharing allows the model to generalize across sequences of varying lengths while maintaining temporal consistency.

Feedforward and Recurrent Connections

In an RNN, two types of connections operate simultaneously:

- **Standard (feedforward) connections** propagate activations from one layer to the next within the same time step.
- **Recurrent connections** transmit information from the hidden state H_{t-1} at one time-step to the hidden state H_t at the next.

Through this mechanism, the hidden state acts as a form of memory, capturing contextual information from earlier elements in the sequence and influencing future predictions.

Backpropagation Through Time

The gradient computation in Recurrent Neural Networks involves performing a forward propagation pass through the unrolled network, followed by a backward propagation pass. Due to the inherently sequential nature of recurrent computation, the runtime complexity is $O(\tau)$, where τ denotes the length of the input sequence. This computation cannot be parallelized across time steps, as each time step depends on the hidden state from the previous one.

During the forward pass, the hidden states at each time step must be stored so that they can be reused during the backward pass. As a result, the memory requirement is also $O(\tau)$. The application of the backpropagation algorithm to the unrolled RNN computation graph is known as Backpropagation Through Time (BPTT).

Merits of Recurrent Neural Networks:

- **Sequential Memory:**
RNNs retain information from previous inputs making them ideal for time-series predictions where past data is crucial. This makes them useful for the tasks such as language modelling, where the meaning of the word depends on the context in which it appears.
- **Variable-length sequences:**
RNNs are designed to handle the input sequences of variable length, which makes them well-suited for tasks such as speech recognition, natural language processing, and time-series analysis.
- **Parameter Sharing:**
RNNs share the same set of parameters across all time steps, which reduces the number of parameters that need to be learnt and can lead to better generalization.

Demerits of Recurrent Neural Networks:

- **Vanishing Gradient:**
During backpropagation, gradients diminish as they pass through each time step leading to minimal weight updates. This limits the RNN's ability to learn long-term dependencies which is crucial for tasks like language translation.
- **Exploding Gradient:**
Sometimes gradients grow uncontrollably causing excessively large weight updates that destabilize training, which leads to the problem of exploding gradient in RNNs.
- **Lack of Parallelism:**
RNNs are inherently sequential, which makes it difficult to parallelize the computation. This limits the speed and scalability of the network compared to architectures like Transformers.

Code and Result

Block 1: Import Libraries & Load Dataset

INPUT

```

import torch

import torch.nn as nn

import numpy as np

import matplotlib.pyplot as plt

plt.rcParams.update({

    "figure.figsize": (8, 5),

    "axes.grid": True,

    "lines.linewidth": 2,

    "font.size": 11

})

# Download Tiny Shakespeare dataset

!curl -L -o tiny_shakespeare.txt https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt

with open("tiny_shakespeare.txt", "r", encoding="utf-8") as f:

    text = f.read()

print("Dataset loaded successfully")

```

OUTPUT:

```

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 1115k  100 1115k    0     0  1234k      0 --:--:-- --:--:-- --:--:-- 1234k

```

Dataset loaded successfully

Block 2: Dataset Preprocessing

INPUT

```
n = len(text)
```

```

train_text = text[:int(0.9*n)]

val_text = text[int(0.9*n):int(0.95*n)]

test_text = text[int(0.95*n):]

print("Train:", len(train_text))

print("Val :", len(val_text))

print("Test :", len(test_text))

# Create character vocabulary and mappings

chars = sorted(list(set(text)))

vocab_size = len(chars)

char2idx = {ch:i for i,ch in enumerate(chars)}

idx2char = {i:ch for i,ch in enumerate(chars)}

def text_to_ints(s):

    return np.array([char2idx[c] for c in s], dtype=np.int32)

train_ids = text_to_ints(train_text)

val_ids = text_to_ints(val_text)

test_ids = text_to_ints(test_text)

print("Vocabulary size:", vocab_size)

```

OUTPUT:

Train: 1003854

Val : 55770

Test : 55770

Vocabulary size: 65

Block 3: Hyperparameters Setup

INPUT

epochs = 15

learning_rate = 0.001

=====

embed_size = 128

batch_size = 64

seq_length = 100

num_layers = 2

hidden_size = 256

def get_batches(data, batch_size, seq_length):

 n_batches = len(data) // (batch_size * seq_length)

 data = data[:n_batches * batch_size * seq_length]

 x = data.reshape(batch_size, -1)

 y = np.roll(x, -1, axis=1)

 return x, y

OUTPUT:

Parameter	Value
Epochs	15
Learning Rate	0.001
Embedding Size	128
Batch Size	64
Sequence Length	100
Number of Layers	2
Hidden Size	256

Block 4: Defining the RNN Model

INPUT

```

class CharRNN(nn.Module):

    def __init__(self, vocab_size, embed_size, hidden_size, num_layers):

        super().__init__()

        self.embed = nn.Embedding(vocab_size, embed_size)

        self.rnn = nn.RNN(embed_size, hidden_size, num_layers, batch_first=True)

        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden):

        x = self.embed(x)

        out, hidden = self.rnn(x, hidden)

        out = self.fc(out)

        return out, hidden

model = CharRNN(vocab_size, embed_size, hidden_size, num_layers)

print(model)

```

OUTPUT:

```

CharRNN(
  (embed): Embedding(65, 128)
  (rnn): RNN(128, 256, num_layers=2, batch_first=True)
  (fc): Linear(in_features=256, out_features=65, bias=True)
)

```

Block 5: Define Training Function

INPUT

```

def train_model(model, train_ids, val_ids):

    criterion = nn.CrossEntropyLoss()

```

```
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

```
train_losses, val_losses = [], []
```

```
for epoch in range(epochs):
```

```
    # ----- Training -----
```

```
    model.train()
```

```
    x, y = get_batches(train_ids, batch_size, seq_length)
```

```
    hidden = None
```

```
    total_loss = 0
```

```
    num_batches = 0
```

```
    for i in range(0, x.shape[1], seq_length):
```

```
        inputs = torch.tensor(
```

```
            x[:, i:i+seq_length], dtype=torch.long
```

```
        )
```

```
        targets = torch.tensor(
```

```
            y[:, i:i+seq_length], dtype=torch.long
```

```
        )
```

```
        optimizer.zero_grad()
```

```
        if hidden is not None:
```

```
            hidden = hidden.detach()
```

```
        outputs, hidden = model(inputs, hidden)
```

```
        loss = criterion(
```

```
            outputs.view(-1, vocab_size),
```

```
            targets.view(-1)
```

```
        )
```

```

        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), 5)

        optimizer.step()

        total_loss += loss.item()

        num_batches += 1

train_losses.append(total_loss / num_batches)

# ----- Validation -----

model.eval()

with torch.no_grad():

    vx, vy = get_batches(val_ids, batch_size, seq_length)

    vhidden = None

    vloss = 0

    v_batches = 0

    for i in range(0, vx.shape[1], seq_length):

        vin = torch.tensor(

            vx[:, i:i+seq_length], dtype=torch.long

        )

        vtar = torch.tensor(

            vy[:, i:i+seq_length], dtype=torch.long

        )

        if vhidden is not None:

            vhidden = vhidden.detach()

        out, vhidden = model(vin, vhidden)

        vloss += criterion(

```

```

        out.view(-1, vocab_size),

        vtar.view(-1)

    ).item()

    v_batches += 1

    val_losses.append(vloss / v_batches)

    print(

        f"Epoch {epoch+1}/{epochs} | "

        f"Train Loss: {train_losses[-1]:.4f} | "

        f"Val Loss: {val_losses[-1]:.4f}"

    )

    return train_losses, val_losses

```

OUTPUT:

✓ train_model function defined successfully

Block 6: Run Training

INPUT

```
train_losses, val_losses = train_model(model, train_ids, val_ids)
```

OUTPUT:

Epoch 1/15 | Train Loss: 2.2668 | Val Loss: 1.9616

Epoch 2/15 | Train Loss: 1.7810 | Val Loss: 1.8115

Epoch 3/15 | Train Loss: 1.6316 | Val Loss: 1.7306

Epoch 4/15 | Train Loss: 1.5502 | Val Loss: 1.6800

Epoch 5/15 | Train Loss: 1.4979 | Val Loss: 1.6450

Epoch 6/15 | Train Loss: 1.4605 | Val Loss: 1.6196

Epoch 7/15 | Train Loss: 1.4319 | Val Loss: 1.6002
Epoch 8/15 | Train Loss: 1.4090 | Val Loss: 1.5848
Epoch 9/15 | Train Loss: 1.3902 | Val Loss: 1.5722
Epoch 10/15 | Train Loss: 1.3742 | Val Loss: 1.5618
Epoch 11/15 | Train Loss: 1.3603 | Val Loss: 1.5534
Epoch 12/15 | Train Loss: 1.3482 | Val Loss: 1.5465
Epoch 13/15 | Train Loss: 1.3371 | Val Loss: 1.5407
Epoch 14/15 | Train Loss: 1.3270 | Val Loss: 1.5360
Epoch 15/15 | Train Loss: 1.3178 | Val Loss: 1.5327

Block 7: Plot Training Loss

INPUT

```
epochs_range = range(1, epochs + 1)

plt.figure()

plt.plot(epochs_range, train_losses, label="Train Loss", marker="o")

plt.plot(epochs_range, val_losses, label="Validation Loss", marker="o")

plt.xlabel("Epoch")

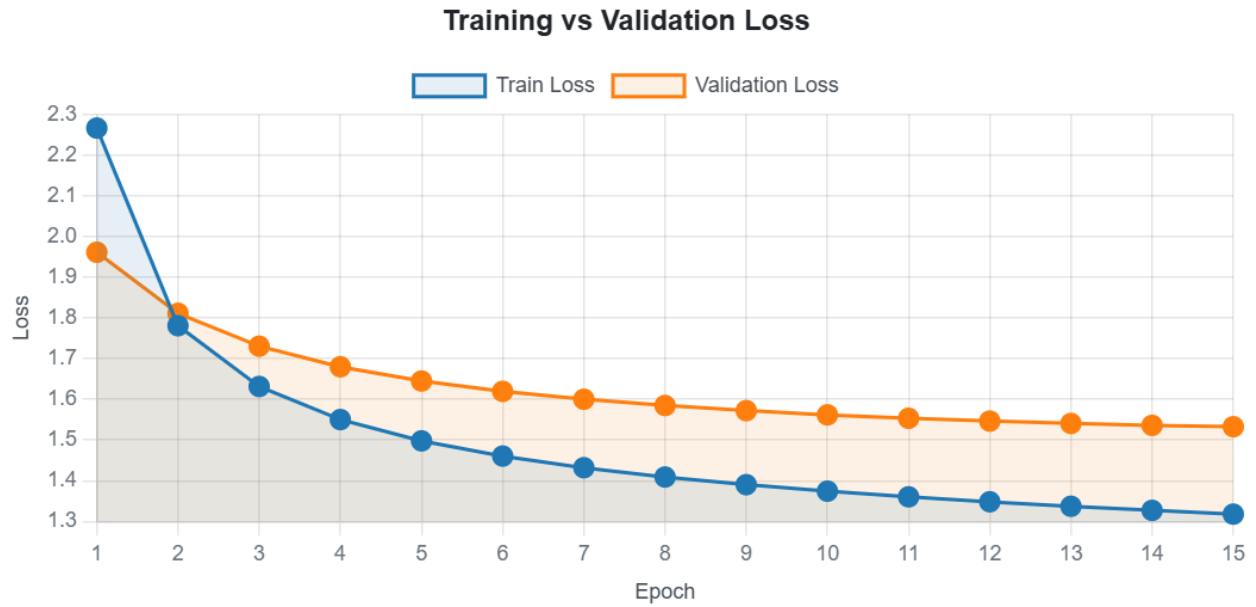
plt.ylabel("Loss")

plt.title("Training vs Validation Loss")

plt.legend()

plt.show()
```

OUTPUT:



Block 8: Text Generation

Starting Text: ROMEO: HELLO!

INPUT

```
def generate_text(model, start_text, length=500):  
  
    model.eval()  
  
    hidden = None  
  
    generated = start_text  
  
    # Prime the RNN with the starting text  
  
    for ch in start_text[:-1]:  
  
        x = torch.tensor([[char2idx[ch]]])  
  
        _, hidden = model(x, hidden)  
  
    # Start generation from the last character  
  
    inp = torch.tensor([[char2idx[start_text[-1]]]])  
  
    for _ in range(length):
```

```
        out, hidden = model(inp, hidden)

        probs = torch.softmax(out[:, -1, :], dim=-1)

        char_idx = torch.multinomial(probs, 1).item()

        generated += idx2char[char_idx]

        inp = torch.tensor([[char_idx]])

    return generated

print(generate_text(model, "ROMEO: HELLO! "))
```

OUTPUT:

 Generated Shakespeare Text:

ROMEO: HELLO! Let God,' my brow-timp it thee enforcey thyself in obey daity, this fair of her
myitie's sovereign, what cannot free four,

In bidieve there: Lord hath with flowers and my sovereign, I speak you hear,

Is the side be

noble pulged

For strengthement spice:

Attend your Edrite,

Catsire, ere us:

Nay does whething exter awhile

Lord Stood of my flies?

First Watchul,

Then, the accusing royal lawf.

First Murderer:

My lord, yet, it be all the worship.

EDWARD:

Deven and tell me like under his nest, and I
